

AI Usage Report

Table of Contents

- [AI Usage Report](#)
 - [Table of Contents](#)
 - [Kim Hằng's AI Usage:](#)
 - [Khánh Linh's AI Usage:](#)
 - [Gia Phúc's AI Usage:](#)
 - [Thiên Phước's AI Usage:](#)
 - [Minh Thư's AI Usage:](#)
 - [Appendix: AI Usage Proof](#)
 - [Kim Hằng](#)
 - [Khánh Linh](#)
 - [Gia Phúc](#)
 - [Thiên Phước](#)
 - [Minh Thư](#)
-

Kim Hằng's AI Usage:

Prompt 1

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Review a Mermaid Use Case Diagram for the "Self-Study Dashboard" subsystem based on two reference articles covering common Use Case Diagram mistakes and how to write effective Use Case Specifications.
- **Prompt used:** "dựa vào hai nguồn trên [thinhnotes.com – Use Case Diagram và 5 sai lầm thường gặp; tinhnotes.com – Viết đặc tả Use Case sao đơn giản nhưng hiệu quả], hãy review đoạn code mermaid này" ("Based on the two sources above, please review this Mermaid diagram code") [with the original Mermaid code attached, containing 11 use cases (UC1–UC11) and include/extend/generalization relationships between the Learner actor and the Self-Study Dashboard system]
- **AI-generated content:**
 - Identified naming issues (UC6, UC7 used noun phrases instead of the Verb + Noun convention; UC1, UC2 had overly long names with parenthetical notes).
 - Flagged misuse of the `<<include>>` relationship to decompose a screen-navigation flow (How) instead of extracting genuinely reusable functionality (What), and pointed out that the include direction was logically backwards (UC1 including UC2, UC2 including UC3).
 - Recommended merging UC9–UC10–UC11 (Track Progress → View % → Calculate %) into a single UC, moving the real-time percentage calculation into a Business Rule/NFR of the specification instead of a separate UC.
 - Pointed out that the `<<extend>>` relationship between UC8 and UC9 lacked an Extension Point and was semantically mismatched (closer to a Trigger/Post-condition than an extend), with an example of a correctly-used extend relationship.
 - Identified a conflict between a direct actor association (Learner → UC3) and an include relationship (UC2 include UC3) reaching the same UC.

- Noted that 11 UCs exceeded the recommended guideline and that UCs were inconsistently mixed across requirement levels (business-level vs. implementation-level).
- **Student's work and validation:**
 - Supplied the original Mermaid diagram and the two reference articles as the basis for comparison.
 - Cross-checked each point raised by the AI against the source articles to confirm accuracy (e.g., verifying the definitions of include/extend and UC naming conventions).
 - Selectively applied the suggested revisions to the final UML diagram submitted for the Self-Study Dashboard specification (PA3-2026), adjusting further to fit the module's actual scope.

Prompt 2

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Get a deeper explanation of the "Use Case Diagram mistaken for Workflow" mistake identified in the previous review, and concrete guidance on how to restructure the Studify Self-Study Dashboard UC diagram correctly.
- **Prompt used:** "nói kĩ hơn về việc nhầm UC case diagram thành workflow, nên sửa như thế nào cho đúng?" ("Explain in more detail the mistake of confusing a Use Case Diagram with a Workflow, and how it should be corrected")
- **AI-generated content:**
 - Clarified the core distinction: a Use Case Diagram answers "WHAT can the Actor achieve" (independent goals), not "HOW/in what click-by-click order" — the latter belongs in the Basic/Alternative Flow of the Use Case Specification, or in a BPMN/Activity Diagram for cross-UC process views.
 - Listed 4 diagnostic signs of a diagram that is actually a workflow in disguise (include chains matching screen-click order; UC's that "can't happen" if a middle step is removed; UC names that read like screen/section titles rather than goals; UC's with no reuse potential).
 - Introduced a "stopping-point test" — asking whether the Learner has achieved a complete, independent value/goal if they stopped at that exact step — and applied it to distinguish true UCs ("Study a Lesson", "Track Learning Progress") from mere steps ("View Lesson Detail").
 - Proposed a concrete 4-step fix for the Studify diagram: (1) list the Learner's true top-level goals (3 root UCs: follow the roadmap, take a quiz, track progress); (2) collapse the roadmap → lesson-detail → theory screen sequence into the Basic Flow of a single "Follow the Learning Roadmap" UC; (3) keep "Take Quiz" as an independent UC since it's reusable/callable on its own, with Multiple-Choice/Fill-in-Blank kept as legitimate generalizations; (4) move "real-time completion % calculation" into a Post-Condition/Business Rule of "Track Learning Progress" instead of a separate UC.
- **Student's work and validation:**
 - Prompted for a deeper explanation after the initial review flagged this specific mistake, to fully understand the underlying UML principle before applying it.
 - Cross-checked the "stopping-point test" and the 4 diagnostic signs against the reference articles to confirm they matched the source material's intent rather than being invented reasoning.
 - Used the 4-step restructuring method to rebuild the actual Self-Study Dashboard UC diagram submitted for PA3-2026, adapting the root-goal list and Basic Flow content to match the real scope of the module rather than copying the AI's version verbatim.

Prompt 3

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Get concrete, Studify-specific examples of correctly-applied <<extend>> and <<generalization>> relationships to reinforce the UML concepts covered in the earlier reviews.
- **Prompt used:** "cho ví dụ về extend và generalization trong các use case này" ("Give examples of extend and generalization within these use cases")
- **AI-generated content:**
 - Three <<extend>> examples grounded in Studify's actual scope, each with an explicit Extension Point: "View Review Suggestions" extending "View Quiz Result" (Extension Point: quiz score < 70%); "Speaking Practice" extending "Study Lesson" (Extension Point: lesson has a Speaking module, tying back to the earlier Azure Pronunciation API work); and "Receive Study Reminder" extending "Track Learning Progress" (Extension Point: inactive > 3 days).
 - Reiterated that every extend relationship must show its Extension Point directly on the diagram, unlike the original UC8→UC9 extend which had none.
 - Three <<generalization>> examples: the already-used Multiple-Choice/Fill-in-Blank as children of "Take Quiz"; a new Login example (Email Login / Google Login as children of "Login", matching Studify's existing JWT + Passport.js setup) with a note that this relationship is often left out of the diagram and handled in the specification's flows instead; and an Actor-level example (Free Learner / Premium Learner as children of Learner) explicitly framed as a future possibility, not a current requirement.
 - Explained the inheritance mechanic behind generalization — a child UC automatically inherits any include relationship of its parent without needing to redraw it.
 - Provided a summary comparison table contrasting Extend vs. Generalization across four dimensions: essence, whether it's optional, what must accompany it (Extension Point vs. none), and Studify-specific examples.
- **Student's work and validation:**
 - Requested examples applied specifically to the Studify use cases already under discussion, rather than generic UML examples.
 - Verified the extend/generalization definitions and the inheritance behavior against the reference articles to confirm correctness.
 - Selected which examples were realistic for the current project scope (e.g., recognized that Actor-level generalization for Free/Premium Learner was not yet applicable) versus which were forward-looking design notes, and used this judgment to decide what to actually include in the PA3-2026 UML specification versus what to leave as future considerations.

Prompt 4

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Start syncing the lesson data displayed under the Studify roadmap with the backend — the level badge had already been synced via the placement test API, but the lesson nodes underneath were still showing hardcoded mock data.
- **Prompt used:** "ok giờ tôi muốn đồng bộ phần lesson ở phía dưới roadmap này, level thì đã đồng bộ từ placement test rồi" ("I now want to sync the lesson section below this roadmap — the level part is already synced from the placement test")
- **AI-generated content:** Observed that the roadmap UI was currently rendering "Lesson 1, 2, 3..." from hardcoded mock data rather than real backend data, and explained that before proposing any fix, it needed to see exactly where the frontend was currently pulling its lesson data from. Asked the student

to paste the relevant frontend file (e.g. `RoadmapPage.jsx` or an equivalently named component) so the actual data-fetching logic could be inspected directly instead of guessed at.

- **Student's work and validation:** Framed the specific integration gap (level synced, lessons not yet synced) based on their own understanding of what had and hadn't been wired up in the codebase, and located and provided the requested frontend file in the next message so the AI could trace the real code path rather than work from assumptions.

Prompt 5

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Confirm the exact shape of the data the roadmap should eventually consume, using a real sample `roadmap.json`, and get a precise read on whether the current frontend code was already fetching that data from the user or still working off static mock data.
- **Prompt used:** "ok vậy đây là mẫu roadmap.json mà web sẽ lấy từ người dùng, cái bây giờ tôi cần là biết code hiện tại có lấy data từ người dùng -> thu nhập nó theo dạng này -> sau đó xuất ra UI" ("Here's a sample roadmap.json the web app will get from the user — what I need now is to know whether the current code takes user data, shapes it into this format, and then renders it to the UI") [with a `roadmap.json` sample containing levels and level ranges pasted in]
- **AI-generated content:** Inspected the provided frontend code and reported that it was importing lesson/node data directly from a hardcoded JSON mock file via `import roadmapData from '../.../mocks/roadmap.json'`, meaning none of the lesson content was actually derived from the user or backend at all — it was static regardless of who was logged in or what their progress was.
- **Student's work and validation:** Supplied a real, project-specific `roadmap.json` sample as the target data contract rather than letting the AI assume a generic shape, and used the confirmation that lessons were still fully mocked as the trigger to plan out the real integration work themselves.

Prompt 6

- **Tool:** Claude (Sonnet 5)
- **Purpose:** Get a full picture of which parts of the roadmap feature were already wired to real data versus still mocked, what the correct end-to-end data flow should look like, and what concretely was blocking the sync, in order to plan the implementation.
- **Prompt used:** Continued from the same exchange as Prompt 05, digging further into the trace of the roadmap data flow.
- **AI-generated content:** Clarified that only the level badge fields (`assignedLevel`, `currentLevelTitle`) were backed by a real API call to `/placement-test/my-roadmap`, while all lesson/node data was still coming from the mock JSON with no connection to user data at all. Laid out the correct end-to-end flow that would be needed instead: placement test result → `assignedLevel` persisted somewhere durable → `GET /roadmap` called with the auth token → backend reads the user's `assignedLevel` together with their `UserProgress` → backend returns roadmap views annotated with the correct per-lesson status (completed/active/locked) → frontend renders that instead of the mock JSON. Alongside this, it flagged three concrete blockers found in the current implementation: `RoadmapService` was reading from `roadmap_levels/roadmap_lessons` tables in the database that were currently empty, the `assignedLevel` value obtained after the placement test was only being kept in in-memory storage (`userLevelStorage`) and would be lost on server restart, and the frontend had no API call at all wired up for fetching roadmap nodes. It also outlined the fastest viable direction to unblock this: drop the DB-backed `RoadmapService` approach in favor of reading lesson structure from

a static `lesson.json` combined with the real `UserProgress` table in the database, deriving each lesson's status from that combination.

- **Student's work and validation:** Used the AI's trace as a diagnostic tool to fully understand where the current implementation was broken and why, rather than as a source of code - the three blockers and the proposed data flow were used purely as a reference to design and implement the actual sync logic (persisting `assignedLevel`, wiring the roadmap API call, and replacing the mock JSON) themselves, consistent with the project's rule of using AI only for consultation, not code generation.

Screenshots/Chat History: *See Appendix*

Khánh Linh's AI Usage:

Prompt 1

- **Tool:** ChatGPT
- **Purpose:** Understand how the answer-submission API for the lesson/exercise grading feature should be designed
- **Prompt used:** "Explain how an API for submitting answers should work, including the difference between submitting a single answer and submitting a whole practice session."
- **AI-generated content:** An explanation of the grading flow, distinguishing single-answer submission vs. full-session submission, and how controller/service/question data interact.
- **Student's work and validation:** Used the explanation to design the actual grading API structure in the STUDIFY backend; cross-checked the suggested flow against the project's existing controller/service pattern before implementing.

Prompt 2

- **Tool:** ChatGPT
- **Purpose:** Understand how frontend code should interpret different API response structures
- **Prompt used:** "How should frontend code interpret API responses such as `data.questions` versus `data.data.questions`?"
- **AI-generated content:** An explanation of common response-wrapping patterns and how nested response structures affect how frontend state should be set.
- **Student's work and validation:** Verified the actual response shape returned by the backend endpoint before writing the frontend state-handling code, to avoid assuming the wrong structure.

Prompt 3

- **Tool:** ChatGPT
- **Purpose:** Understand the avatar upload architecture across backend and frontend
- **Prompt used:** "Explain the roles of Multer, static assets, controller/service/model, database mapping, JWT user ownership, and frontend fallback avatars in an avatar upload feature."
- **AI-generated content:** An explanation of how uploaded files flow from Multer through storage, how they are linked to a user record via JWT, and how the frontend falls back to a default avatar when none exists.
- **Student's work and validation:** Applied the explanation to trace and confirm the existing avatar upload implementation in the STUDIFY codebase rather than writing new code from the AI's answer.

Prompt 4

- **Tool:** ChatGPT
- **Purpose:** Understand the forgot-password and OTP authentication flow
- **Prompt used:** "How does password-reset OTP work, including controller/service responsibilities, DTO validation, OTP hashing, expiration time, deletion after use, Nodemailer, Gmail 2-Step Verification, and App Passwords?"
- **AI-generated content:** An explanation of the end-to-end OTP security flow, covering hashing, expiration, single-use deletion, and the Gmail/Nodemailer email-sending requirements.
- **Student's work and validation:** Used the explanation to review and validate the security logic already implemented in the password-reset feature, checking that hashing and expiration handling matched the recommended practice.

Prompt 5

- **Tool:** ChatGPT
- **Purpose:** Determine correct grading behavior for different question types
- **Prompt used:** "How can multiple-choice answers be automatically graded while written answers are treated differently and shown for review instead of automatically marked correct or incorrect?"
- **AI-generated content:** An explanation of the distinction between auto-gradable objective questions and written questions requiring manual/deferred review.
- **Student's work and validation:** Used this distinction to define the grading rules applied in the STUDIFY quiz feature, then implemented and tested the logic separately for each question type.

Screenshots/Chat History: *See Appendix*

Gia Phúc's AI Usage:

Prompt 1

- **Tool:** Copilot, GitHub via Visual Studio Code
- **Purpose:** Perform a rapid UI check across the codebase to identify visual bugs, broken components, and rendering defects without manually clicking through every individual page.
- **Prompt used:** List all the prominent UI errors that can currently be seen
- **AI-generated content:** Generated a list of detected visual defects and layout inconsistencies, along with their corresponding page.
- **Student's work and validation:** Manually navigate to each reported page to verify the errors flagged. Filtered out any false reports and created a plan to fix valid layout and styling issues.

Prompt 2

- **Tool:** Google Gemini
- **Purpose:** Analyze functional requirement F3.4 (Shared File Repository) within the Virtual Study Room module to get an idea of user interactions, file format restrictions, and role-based permissions.
- **Prompt used:** Phân hệ 3: Không gian Học nhóm (Virtual Study Room) giới hạn từ 2 - 5 người

F3.1: Quản lý Phòng học/Nhóm học (Group Management) Tạo nhóm học tập mới (Hệ thống tự động cấp Mã Code nhóm). Tham gia nhóm học tập thông qua Mã Code (Join via Code).

F3.2: Phân quyền Thành viên (Role Authorization) Quyền Leader (Trưởng nhóm): Quản lý nhóm, giao việc, đăng tài liệu. Quyền Member (Thành viên): Xem thông tin, nhận nhiệm vụ, tải tài liệu.

F3.3: Quản lý & Giao Nhiệm vụ (Task Assignment) Đối với Leader: Tạo lịch học, tạo nhiệm vụ, đặt hạn chót (Deadline) cho nhóm. Đối với Member: Nhận và xem danh sách nhiệm vụ cần làm thông qua một Widget thông báo hiển thị trực tiếp tại Dashboard cá nhân khi đăng nhập.

F3.4: Kho lưu trữ tài liệu chung (Shared File Repository) Đăng tải tài liệu học tập bổ sung (Hỗ trợ định dạng PDF, hình ảnh). Xem và tải về (Download) tài liệu do các thành viên khác chia sẻ.

F3.5: Real-time chat Các thành viên có thể thảo luận với nhau trong một khung chat chung.

Explain the flow of going to the group's page and accessing the file repository and download/upload files

- **AI-generated content:** Generated a 4-step user interaction flow detailing group navigation, file repository views, upload constraints/validation (PDF/image formats, storage limits), preview/download mechanics, and real-time chat notification triggers.
- **Student's work and validation:** Verified the process steps to define the UI structure (Shared Files tab, previews , upload UI).

Prompt 3

- **Tool:** Google Gemini
- **Purpose:** Check a Mermaid UC diagram with a feedback to identify any violations and align the new model with standard UML practices.
- **Prompt used:** can you check if my UC mermaid model fit my professor's feedback? anything that need changing? flowchart LR %% Actors Learner(["Learner"]) AI["AI Engine"]

```
%% System Boundary subgraph AI_Speaking_Assistant ["AI Speaking Assistant"] UC51(("UC5.1: Speech recognition")) UC52(("UC5.2: Generate appropriate reply")) UC53(("UC5.3: Evaluate performance on specific criteria")) UC54(("UC5.4: Provide guidance on how to improve")) end
```

```
%% Primary Interactions Learner --> UC51 Learner --> UC52
```

```
%% Relationships UC52 -. "<>" .-> UC51 UC53 -. "<>" .-> UC51 UC54 -. "<>" .-> UC53
```

```
%% Secondary Actor Interactions UC51 --> AI UC52 --> AI UC53 --> AI UC54 --> AI can you check this feedback if my mermaid model fit the criticise? flowchart LR %% Actors Learner(["Learner"]) AI["AI Engine"]
```

```
%% System Boundary subgraph AI_Speaking_Assistant ["AI Speaking Assistant"] UC51(("UC5.1: Speech recognition")) UC52(("UC5.2: Generate appropriate reply")) UC53(("UC5.3: Evaluate performance on specific criteria")) UC54(("UC5.4: Provide guidance on how to improve")) end
```

```
%% Primary Interactions Learner --> UC51 Learner --> UC52
```

```
%% Relationships UC52 -. "<>" .-> UC51 UC53 -. "<>" .-> UC51 UC54 -. "<>" .-> UC53
```

```
%% Secondary Actor Interactions UC51 --> AI UC52 --> AI UC53 --> AI UC54 --> AI
```

- **AI-generated content:** Analyzed the diagram against standard UML rules, identified anti-patterns such as functional decomposition and overuse of <> relationships. Provided concrete refactoring suggestions to group low-level mechanics into goal-oriented use cases.
- **Student's work and validation:** Reviewed the critique against the current diagram. Verified which suggestions addressed the correct design flaws, updated the Mermaid diagram.

Screenshots/Chat History: *See Appendix*

Thiên Phước's AI Usage:

Prompt 1

- **Tool:** Antigravity AI (Gemini / Claude via IDE assistant)
- **Purpose:** Review and improve the C4 Model Level 3 (Component Diagram) document — identify missing container descriptions, inappropriate feature selection, and incomplete component diagrams; then generate a fully revised document that accurately reflects the actual source code.

- **Prompt used:**

Đọc kĩ nhiệm vụ và yêu cầu dưới đây, có vấn đề gì về file C4 Model - Level 3 (Component Diagram).md không? Thiếu sót, thiếu diagram cho chức năng quan trọng nào không?

C4 Model - Level 3 (Component Diagram): draw Component diagrams following the C4 Model for both the frontend and backend containers. Each diagram zooms into a container to show the major components inside it, their responsibilities, and the interactions between them. You are not required to draw Level 3 diagrams for every feature; instead, focus on the most important features that best represent the internal structure of each container.

Requirements: All diagrams must be drawn using Mermaid format. You may use standard Mermaid flowchart/graph syntax with C4-style labeling (Mermaid's experimental C4 syntax is not required). For each container, provide a written description of: Its responsibility and the services it provides. The technology/framework used. How it communicates with other containers (e.g., HTTP/HTTPS, WebSocket, database connection). For each component in the Level 3 diagram, describe its responsibility and its relationships with other components. Warning: The architecture diagrams must accurately reflect your actual implementation at the time of submission. Any inconsistency between the documented architecture and the source code will result in a grade penalty.

As you continue implementing features, you must keep the architecture document up to date.

- **AI-generated content:** The AI reviewed the existing Level 3 diagram, identified two key problems: (1) missing Container Descriptions (Responsibility, Technology, Communication) required by the spec; (2) only Authentication was documented, which is too generic and does not showcase the system's unique architecture. The AI then read the actual source code across the entire codebase and rewrote the complete Level 3 document covering two features — **Authentication & Onboarding (Frontend)** and **Authentication & Placement Test (Backend)** — with accurate Mermaid diagrams and full component descriptions for all 11 real components found in the code.

- **Student's work and validation:** Reviewed the AI's analysis and confirmed the identified issues were correct. Verified that all component names, file paths, API endpoints, and inter-component relationships in the generated diagrams match the actual source code (e.g., `useAuthStore.ts`, `PlacementTestService`, `JwtGuard`, `ProgressService`). Validated the correctness of the Smart Onboarding algorithm description and the MailService / Gmail SMTP dependency.

Screenshots/Chat History: *See Appendix*

Prompt 2

- **Tool:** Antigravity AI (Gemini / Claude via IDE assistant)
- **Purpose:** Analyze TA's feedback on PA3, refactor the Module 4 Use Case Model (Mermaid) to resolve "Functional Decomposition" (workflow abuse), merge fragmented Use Case Specifications, and correct duplicate IDs and image references.
- **Prompt used:**

trưởng nhóm giao là mỗi thành viên tự sửa lại phần của mình đã làm trong PA3, trong đó thì mình đã làm Usecase cho module 4 ở phần D. Bạn phân tích kỹ xem mình có cần sửa gì dựa vào feedback của thầy không?

- **AI-generated content:** The AI reviewed the existing `report.md` and 10 fragmented specification files in `Module_4`. It identified that the use cases were inappropriately broken down into workflow steps (`include` abuse) and had duplicate IDs. It generated an implementation plan and then refactored the 10 files into 3 comprehensive specifications (`M4-UC1`, `M4-UC2`, `M4-UC3`), updated the Mermaid diagram, and correctly linked all UI prototype images.
- **Student's work and validation:** Reviewed the AI's analysis and the refactored use cases to ensure they adhere to UML standards and correctly represent the intended Module 4 functionality. Verified that the image paths were correct and the Mermaid diagram rendered properly.

Screenshots/Chat History: *See Appendix*

Minh Thu's AI Usage:

Prompt 1

- **Tool:** Gemini 3.6 Thinking
- **Purpose:** Debugging Pomodoro timer state logic in TypeScript/Zustand (fixing a 2-second countdown skip issue and updating default mode reset behavior).
- **Prompt used:** "kiểm tra xem vì sao khi bắt đầu, nó bị trừ 2s? tức là từ 1 phút, nó không xuống 59s mà xuống thẳng 58s. và t muốn chỉnh sửa thêm: khi ấn refresh timer hay reload gì đấy, nó sẽ tự động quay về mode focus chứ không phải relax" [accompanied by the TypeScript Zustand store code]
- **AI-generated content:**
 - Explanation of root cause: Using `Math.floor` alongside event loop execution delays in `setInterval(1000)` caused `$58.99\text{s}$` to round down immediately to `$58\text{s}$`.
 - Refactored TypeScript code replacing `Math.floor` with `Math.ceil`, increasing interval frequency to `$200\text{ms}$` for smooth accuracy, and modifying `resetTimer` and `getInitialState` to always default to "focus" mode upon reload or reset.

- **Student's work and validation:**

- Provided original application code and pinpointed specific UI/UX bugs (timer skipping from 01:00 straight to 00:58 and unexpected session retention on reload).
- Validated the suggested fix by testing the countdown behavior to ensure smooth decrement from 01:00 $\rightarrow$ 00:59, verifying interval accuracy, and confirming the store correctly resets to "focus" mode on page refresh.

Prompt 2

- **Tool:** Gemini 3.6 Thinking
- **Purpose:** Maintain Pomodoro timer state and browser tab title updates across route transitions (e.g., navigating from /dashboard/pomodoro to /dashboard).
- **Prompt used:** "useEffect() => { ... }) cái này, t muốn khi người dùng di chuyển sang tab khác trong studify (vdu: đường dẫn hiện tại là /dashboard/pomodoro, khi user di chuyển qua đường dẫn /dashboard thì vẫn sẽ hiện giống vậy)"
- **AI-generated content:** Explanation of component unmounting during route changes and a solution architecture converting the custom hook into a global React Context (PomodoroContext.tsx).
- **Student's work and validation:** Provided original useEffect title update code; reviewed the Context approach and decided to explore popup/widget options before committing to implementation.

Prompt 3

- **Tool:** Gemini 3.6 Thinking
- **Purpose:** Debug a React default import vs. named export error.
- **Prompt used:** "Uncaught SyntaxError: The requested module '/src/features/pomodoro/PomodoroTimer.ts' does not provide an export named 'default' (at App.tsx:26:8)"
- **AI-generated content:** Explained default vs. named exports and provided two fixes: adding export default to the component file or using named imports { Pomodoro } in App.tsx.
- **Student's work and validation:** Caught the runtime error in browser dev tools and applied the export/import syntax fix.

Screenshots/Chat History: *See Appendix*

Appendix: AI Usage Proof

Kim Hằng

Prompt 1

<https://thinhnotes.com/chuyen-nghe-ba/use-case-diagram-va-5-sai-lam-thuong-gap/>

<https://thinhnotes.com/chuyen-nghe-ba/viet-dac-ta-use-case-sao-don-gian-nhung-hieu-qua/>

dựa vào hai nguồn trên, hãy review đoạn code mermaid này

```
flowchart LR
  %% Actor
  Learner(["Learner"])
```

Show more

Diagnosed structural flaws in use case diagram design comprehensively

Đọc xong 2 bài viết của anh Thịnh, mình thấy sơ đồ của bạn có khá nhiều điểm đáng chỉnh lại. Dưới đây là review chi tiết theo từng nhóm vấn đề.

1. Đặt tên Use Case (mục 3.1, bài 1)

- UC6: Multiple Choice và UC7: Fill in the Blank — đây là cụm danh từ, không theo format **Verb + Noun**. Nên đổi thành **Answer Multiple-Choice Question**, **Answer Fill-in-the-Blank Question**.
- UC1: View Learning Roadmap (by level A1, A2...) và UC2: View Lesson Details by level — tên hơi dài, kèm chú thích trong ngoặc. Bài viết khuyên tên UC nên ngắn gọn, súc tích; phần "by level A1, A2..." nên đẩy xuống mô tả (Description) trong đặc tả UC, không nhét vào tên trên diagram.

2. Include đang bị dùng sai bản chất — đây là vấn đề lớn nhất

Bài 1 nhấn rất mạnh: Include chỉ nên dùng khi UC được tách ra có khả năng tái sử dụng ở nơi khác, không phải để phân rã chức năng (mục 3.2 — lỗi sai lầm phổ biến số 2).

Nhìn vào chuỗi:

```
UC1 --include--> UC2 --include--> UC3 --include--> UC4/UC5
```

Chuỗi này đang mô tả **luồng click màn hình** (roadmap → chi tiết bài học → học bài → lý thuyết/quiz), tức là trả lời câu hỏi "How" chứ không phải "What" — đúng cái mà bài viết cảnh báo: *"tập trung vào câu hỏi What để tìm ra Use Case, tránh câu hỏi How"*.

Ngoài ra, hướng include cũng đáng ngờ về logic:

- **UC1 include UC2**: nghĩa là để xem được Roadmap thì bắt buộc phải xem xong Chi tiết bài học — ngược đời, vì thực tế là ngược lại (từ Roadmap mới đi vào Chi tiết bài học).
- **UC2 include UC3**: tương tự, "Xem chi tiết bài học" include "Học bài" nghĩa là muốn xem chi tiết thì bắt buộc phải học xong bài — vô lý.

Còn **UC9 include UC10 include UC11** (Track Progress → View % → Calculate %) cũng là kiểu phân rã, không phải include tái sử dụng — vì **UC10**, **UC11** không được dùng lại ở UC nào khác trong diagram.

Cách sửa theo tinh thần bài viết: những bước như "Xem % hoàn thành", "Tính % hoàn thành" nên gộp lại thành 1 UC duy nhất (hoặc note trong Basic Flow của đặc tả), chứ không tách thành 3 UC rồi có quan hệ include dây chuyền. **UC11 (Calculate Completion % real-time)** đặc biệt đáng gộp — đây là hành vi nội bộ hệ thống, không phải tương tác Actor-System theo đúng nghĩa, nên đưa vào Business Rule/NFR của đặc tả UC9 thay vì làm riêng 1 UC (đúng tinh thần bài 2, mục Business Rule/↓).

3. Extend thiếu Extension Point và hướng chưa hợp lý

`UC8 --extend--> UC9` (View Quiz Result extend Track Learning Progress): bài viết nhấn mạnh Extend luôn cần đi kèm **Extension Point** (điều kiện cụ thể khi nào extend xảy ra). Ở đây không có điều kiện nào được nêu, nên người đọc không biết "khi nào" quan hệ này xảy ra.

Về mặt ý nghĩa cũng hơi ngược — "Xem kết quả Quiz" hoàn thành xong mới **cập nhật/kích hoạt** tiến độ học tập, nghĩa là quan hệ ở đây gần với **trigger** (Post-Condition của UC8 = Trigger của UC9) hơn là extend. Extend đúng nghĩa nên dùng cho tình huống optional, ví dụ: "Xem gợi ý ôn tập lại" chỉ extend từ "Xem kết quả Quiz" khi điểm dưới ngưỡng nào đó.

4. Generalization — hướng đúng, nhưng nội dung không nhất quán

`UC6 --> UC5` và `UC7 --> UC5` (con trở về cha) là đúng chiều theo mô tả trong bài viết. Tuy nhiên đây lại quay về vấn đề đặt tên ở mục 1 — sửa tên `UC6`, `UC7` là ổn.

5. Actor association mâu thuẫn với chuỗi include

`Learner --> UC3` (association trực tiếp) nhưng đồng thời `UC2 --include--> UC3` (UC3 chỉ đạt được thông qua include từ UC2). Hai quan hệ này xung đột nhau: nếu UC3 chỉ nên đạt được bằng cách include từ UC2, thì không cần association trực tiếp từ Actor nữa (và ngược lại). Bài viết có nhắc trong comment: Actor chỉ nên nối tới UC mà nó **trực tiếp** kích hoạt (entry point), còn UC nào chỉ đạt được qua include thì không cần vẽ association riêng.

6. Số lượng Use Case và mức độ chi tiết

11 UC trong 1 Boundary là hơi nhiều so với khuyến nghị "trên dưới 10 UC" (mục 3.3), và nhiều UC trong số đó (UC4, UC10, UC11) đang ở **cấp độ chi tiết implementation** (Solution Requirement) trong khi UC1, UC9 lại ở cấp High-level (Business/User Requirement) — đây chính là lỗi "*lệch cấp độ Requirement*" mà bài viết cảnh báo ở mục 3.2.



Đề xuất chỉnh sửa

Prompt 2

nói kĩ hơn về việc nhẫm UC case diagram thành workflow, nên sửa như thế nào cho đúng?

Phân tích sự khác biệt giữa use case và workflow, đề xuất cấu trúc lại

Bản chất của lỗi "biến Use Case Diagram thành Workflow"

Theo bài 1, Use Case Diagram trả lời câu hỏi: "Hệ thống cho phép Actor làm được những gì (WHAT), nhằm đạt mục đích gì" — chứ không trả lời "Actor làm việc đó theo trình tự các bước nào, click màn hình nào trước, màn hình nào sau (HOW)".

Đó là lý do bài viết nhấn: "Khi vẽ Use Case Diagram, tập trung vào câu hỏi What để tìm ra Use Case, tránh câu hỏi How, vì khi đó anh em rất dễ đi vào detail."

Còn trình tự thao tác (How) là việc của Basic Flow / Alternative Flow bên trong bản đặc tả (Use Case Specification) — hoặc nếu muốn nhìn tổng thể quy trình liên UC/liên hệ thống thì dùng BPMN hoặc Activity Diagram (bài 1 có nhắc: "nếu muốn nhìn tổng thể process thì nên dùng activity diagram hoặc BPMN").

Dấu hiệu nhận biết bạn đang vẽ Workflow chứ không phải UC Diagram

1. Các UC nối include với nhau theo đúng **thứ tự thao tác trên màn hình** ($A \rightarrow B \rightarrow C \rightarrow D$), giống hệt cách người dùng bấm chuột từng bước.
2. Nếu xóa 1 UC ở giữa chuỗi, các UC còn lại "không xảy ra được" — vì bản chất chúng chỉ là **bước kế tiếp**, không phải mục tiêu độc lập.
3. Tên UC nghe như tên **màn hình/section** ("Xem chi tiết bài học") hơn là **hành động có mục đích riêng** ("Học theo lộ trình").
4. UC không có khả năng tái sử dụng ở bất kỳ ↓ nào khác trong hệ thống — vì nó chỉ tồn tại để phục vụ đúng 1 chuỗi thao tác đó.

Sơ đồ Studify của bạn dính cả 4 dấu hiệu này ở chuỗi UC1>UC2>UC3>UC4/UC5 và UC9>UC10>UC11 .

Câu hỏi test nhanh: đây có phải là 1 Use Case thật sự không?

Tự hỏi: "Nếu Learner dừng lại đúng ở bước này, họ có đạt được một giá trị/mục đích trọn vẹn nào đó không?"

- "Xem chi tiết bài học" → dừng ở đây thì Learner chưa đạt được gì cả, họ chỉ mới xem thông tin để chuẩn bị học → đây là 1 bước trong flow, không phải UC.
- "Học bài" (hoàn thành lý thuyết + làm quiz) → dừng ở đây Learner đã đạt được mục đích: học xong 1 bài → đây là 1 UC.
- "Theo dõi tiến độ học tập" → Learner mở dashboard xem % hoàn thành → tự thân nó là 1 mục đích độc lập (không phụ thuộc bước nào trước) → đây là 1 UC.

Include chỉ nên tồn tại khi UC con thật sự có thể được gọi lại ở nhiều UC cha khác (bài viết: "chỉ nên tách Use Case khi nó có độ phức tạp lớn và những thứ tách ra được có thể được tận dụng ở các Use Case sau này"), chứ không phải chỉ vì nó "xảy ra trước" UC kia.

Cách sửa cụ thể cho Studify

Bước 1 — Liệt kê "mục đích" thật sự của Learner (không nhìn màn hình, chỉ nhìn goal):

Goal của Learner	UC tương ứng
Biết mình cần học gì, theo thứ tự nào	Học theo lộ trình (roadmap)
Kiểm tra kiến thức đã học	Làm bài kiểm tra
Biết mình đã học được bao nhiêu	Theo dõi tiến độ học tập

→ Chỉ 3 UC "gốc" ở mức goal, không phải 11 UC như trước.

Bước 2 — Dồn các bước điều hướng (roadmap → lesson detail → study theory) vào Basic Flow của UC "Học theo lộ trình":

UC: Học theo lộ trình
Basic Flow:
1. Learner mở lộ trình học theo trình độ (A1, A2...)
2. Hệ thống hiển thị danh sách bài học theo lộ trình
3. Learner chọn 1 bài học
4. Hệ thống hiển thị nội dung lý thuyết rút gọn của bài học
5. Learner học xong lý thuyết, chọn "Làm bài kiểm tra"
→ UC tiếp tục UC: Làm bài kiểm tra

→ Toàn bộ chuỗi UC1>UC2>UC3>UC4 cũ giờ chỉ là các bước (step) 1-4 trong 1 Basic Flow duy nhất — đúng tinh thần bài 2, không cần tách UC.

Bước 3 — "Take Quiz" giữ nguyên là 1 UC (vì nó là 1 mục tiêu độc lập, có thể được gọi lại từ nhiều nơi — ví dụ Learner có thể vào ôn tập và làm lại quiz mà không cần đi qua lộ trình), và Multiple Choice / Fill-in-Blank giữ generalization vì đó là biến thể loại câu hỏi thật sự (không phải bước tuân tự).

Bước 4 — "Tính % hoàn thành real-time" đưa vào Business Rule/Post-Condition của UC "Theo dõi tiến độ học tập", không tách UC riêng:

UC: Theo dõi tiến độ học tập
Post-Condition: Hệ thống tính lại % hoàn thành lộ trình mỗi khi có bài học/quiz mới hoàn thành.

Prompt 3

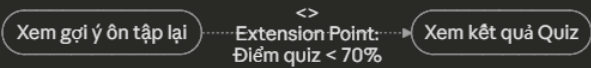
cho ví dụ về extend và generalization trong các use case này

Orchestrated Studify use case examples grounding extend and generalization concepts

Extend trong ngữ cảnh Studify

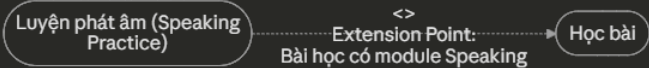
Nhắc lại định nghĩa từ bài viết: Extend = quan hệ **không bắt buộc**, chỉ xảy ra trong 1 hoàn cảnh cụ thể — và luôn nên có **Extension Point** đi kèm để nói rõ điều kiện đó là gì.

Ví dụ 1: Gợi ý ôn tập lại

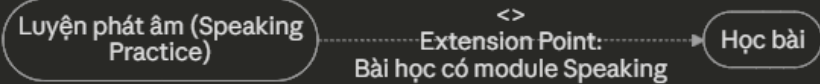


- **Base UC:** Xem kết quả Quiz — xảy ra với mọi Learner sau khi làm quiz.
- **Extend UC:** Xem gợi ý ôn tập lại — chỉ xảy ra **khi** điểm dưới ngưỡng đạt. Learner đạt điểm cao thì UC này **không bao giờ xuất hiện**.
- Đây đúng bản chất Extend: optional + có điều kiện rõ ràng (Extension Point).

Ví dụ 2: Luyện phát âm



Ví dụ 2: Luyện phát âm



- Không phải bài học nào cũng có phần luyện phát âm (theo scope Studify bạn từng làm với Azure Pronunciation API) → chỉ những bài có module speaking mới kích hoạt UC này.

Ví dụ 3: Nhắc nhở học tập

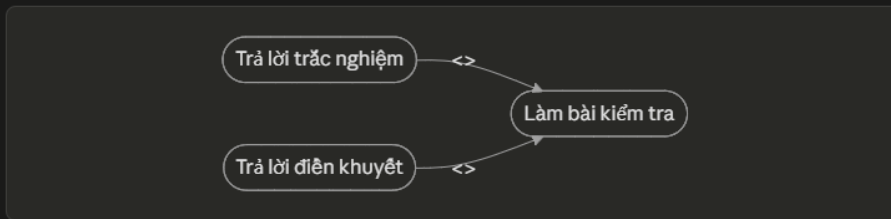


Lưu ý quan trọng khi vẽ Extend: luôn viết rõ Extension Point ngay trên đường nối (như 3 ví dụ trên), đừng để trông như trong bản gốc của bạn (UC8 extend UC9 không có điều kiện) — vì người đọc sẽ không biết "khi nào" nó xảy ra, và rất dễ nhầm với Include.

Generalization trong ngữ cảnh Studify

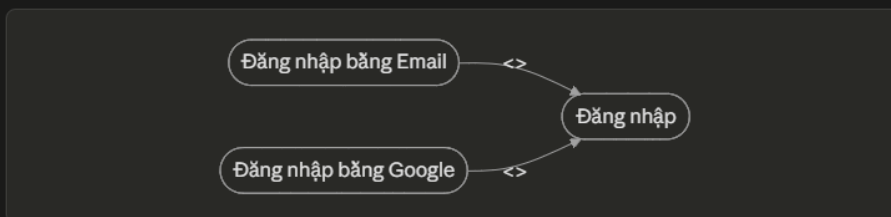
Định nghĩa: quan hệ cha-con, có tính kế thừa — và áp dụng được cho cả Use Case lẫn Actor.

Ví dụ 1: Generalization giữa các UC (loại câu hỏi — đã dùng trong sơ đồ)



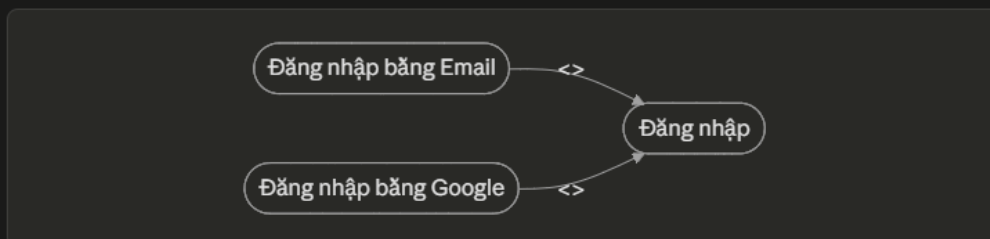
- Trả lời trắc nghiệm và Trả lời diễn thuyết đều là một dạng cụ thể của Làm bài kiểm tra. Nếu Làm bài kiểm tra có include tới Ghi nhận điểm số, thì cả 2 UC con này tự động kế thừa quan hệ include đó dù không cần vẽ lại (đúng như bài viết: "Use Case A có include đến B, thì Use Case A' là con của A cũng sẽ có mối quan hệ Include đến B, mặc dù không được thể hiện trên hình").

Ví dụ 2: Generalization ở Đăng nhập



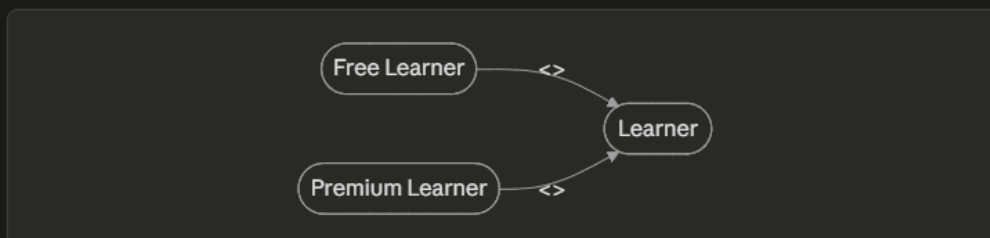
- Khớp với backend Studify hiện tại (JWT + passport.js, có khả năng hỗ trợ social login). Đây là ví dụ kinh điển bài viết đưa ra: "Đăng nhập thì có thể đăng nhập qua số điện thoại, hoặc đăng nhập qua email".

Ví dụ 2: Generalization ở Đăng nhập



- Khớp với backend Studify hiện tại (JWT + Passport.js, có khả năng hỗ trợ social login). Đây là ví dụ kinh điển bài viết đưa ra: "Đăng nhập thì có thể đăng nhập qua số điện thoại, hoặc đăng nhập qua email".
- Lưu ý: bài viết cũng nói rõ đây là relationship ít khi cần vẽ chi tiết trên diagram — nhiều khi chỉ cần 1 UC "Đăng nhập" rồi mô tả các phương thức trong Basic/Alternative Flow của đặc tả là đủ, không nhất thiết phải tách generalization ra hình vẽ (dở rồi UC Diagram).

Ví dụ 3: Generalization ở cấp Actor



- Đây là ví dụ minh họa cho trường hợp Studify sau này phân cấp tài khoản (free/premium) — hiện tại hệ thống bạn đang làm chỉ có 1 loại Learner nên chưa cần vẽ thêm relationship này. Đưa ra để bạn hình dung: nếu Learner (cha) có association tới Học theo lộ trình, thì cả Free Learner và Premium Learner (con) đều kế thừa quyền đó tự động — không cần vẽ lại association riêng cho từng loại.

Điểm khác biệt cốt lõi cần nhớ khi áp dụng vào Studify

	Extend	Generalization
Bản chất	1 UC là phần mở rộng tùy chọn của UC khác	1 UC/Actor là 1 dạng cụ thể của UC/Actor tổng quát hơn
Bắt buộc?	Không — chỉ xảy ra khi thỏa điều kiện	Không áp dụng khái niệm "bắt buộc" — con luôn là 1 biến thể của cha
Cần gì đi kèm	Extension Point (điều kiện)	Không cần điều kiện, chỉ cần tên rõ ràng thể hiện tính "là-một-loại" (is-a)
Ví dụ trong Studify	Gợi ý ôn tập, Luyện phát âm, Nhắc nhở học tập	Trắc nghiệm/Điền khuyết, Đăng nhập Email/Google

Prompt 4



ok giờ tôi muốn đồng bộ phân lesson ở phía dưới roadmap này, level thì đã đồng bộ từ placement test rồi

Jul 29

Roadmap đang hiển thị "Lesson 1, 2, 3..." là data hardcoded. Để đồng bộ với backend, tôi cần biết frontend đang lấy data từ đâu.

Paste file `RoadmapPage.jsx` (hoặc tên tương tự) lên cho tôi xem.

Prompt 5

```
import React, { useState,
useRef, useEffect } from 'r
eact'; import axios from 'a
xios'; import { useAuthSto
re } from '../auth/store/u
seAuthStore'; import ro...
```

PASTED

Thấy rõ vấn đề rồi. Roadmap đang dùng `roadmapData` từ file JSON cứng (`mocks/roadmap.json`) cho phần nodes/lessons, chỉ có level badge mới gọi API.

Cần làm 2 việc:

- 1. Backend — `GET /roadmap` cần trả về đúng format mà frontend đang dùng (`views`, `levels`, `metrics`). `RoadmapService` đã có sẵn logic này nhưng đang đọc từ DB trống.
- 2. Frontend — thay `roadmapData` bằng data từ API.

Trước khi code, tôi cần biết: `roadmaps` và `roadmap_lessons` tables trong DB có data chưa? Vì `RoadmapService` đọc từ DB, nếu trống thì trả về `views: []`.

Chạy lệnh này trong psql hoặc DBeaver:

```
sql
SELECT COUNT(*) FROM "roadmap_levels";
SELECT COUNT(*) FROM "roadmap_lessons";
```

Kiểm tra xong rồi?

Prompt 6

```
{ "levels": ["A1","A2","B1","B
2","C1","C2"], "levelRange
s": { "A1": { "start": 1, "end":
10 }, "A2": { "start": 11, "en
d": 20 }, "B1": { "start": 21,
"end": 30 }, "B2": { "start..."
```

PASTED

ok vậy đây là mẫu roadmap.json mà web sẽ lấy từ người dùng, cái bây giờ tôi cần là biết code hiện tại có lấy data từ người dùng -> thu nhập nó theo dạng này-> sau đó xuất ra UI

Nhìn vào code hiện tại:

Frontend đang import thẳng từ file JSON cứng:

```
javascript
import roadmapData from '../.../mocks/roadmap.json';
```

Chỉ có phần level badge (`assignedLevel`, `currentLevelTitle`) là gọi API thật (`/placement-test/my-roadmap`). Còn toàn bộ nodes/lessons vẫn dùng mock JSON — không lấy data từ người dùng.

Để đồng bộ đúng, luồng cần là:

User làm placement test → assignedLevel lưu đâu đó
 → GET /roadmap (kèm token)
 → Backend đọc assignedLevel của user + UserProgress
 → Trả về views với status đúng (completed/active/locked)
 → Frontend dùng data đó thay mock JSON

Vấn đề hiện tại:

1. `RoadmapService` đọc từ bảng `roadmap_levels` / `roadmap_lessons` trong DB — bảng này trống
2. `assignedLevel` sau placement test lưu trong memory (`userLevelStorage`) — mất khi restart
3. Frontend chưa gọi API cho roadmap nodes

Hướng nhanh nhất — bỏ `RoadmapService` dùng DB, thay bằng đọc từ `lesson.json` + `UserProgress` DB. Cụ thể:

- GET /roadmap nhận `userId` từ JWT token
- Đọc `assignedLevel` từ bảng `PlacementTestResult` (đã lưu vào DB)
- Build views từ `lesson.json` theo level được assign
- Map với `UserProgress` để xác định status từng lesson

Khánh Linh

Prompt 1

The screenshot shows a web browser window with the ChatGPT Plus interface. The chat is titled "Khánh Linh". The main content area displays a TypeScript code snippet for a POST request. The code is as follows:

```

TypeScript

const response = await fetch(
  `http://localhost:3000/learning/lessons/${lessonId}/submit-practice`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      ...(token
        ? {
            Authorization: `Bearer ${token}`,
          }
        : {}),
    },
    body: JSON.stringify({
      answers,
    }),
  },
);

const data = await response.json();

if (!response.ok) {
  throw new Error(
    'Error: ' + response.statusText
  );
}

```

The browser's address bar shows the URL: `chatgpt.com/g/g-p-6a68a9bb684881919c6c8883ffc1d78c/c/6a69b48b-21fc-83ed-b45d-ae5ef19a12bc`. The left sidebar shows a list of pinned chats, including "introSe" and "Phân tích KNN và tham số". The bottom of the interface shows the "Ask ChatGPT" button and a "Medium" dropdown menu.

The screenshot displays the ChatGPT Plus web interface. The browser's address bar shows the URL: `chatgpt.com/g/g-p-6a68a9bb684881919c6c8883ffc1d78c/c/6a69b48b-21fc-83ed-b45d-ae5ef19a12bc`. The left sidebar contains navigation options: 'Scheduled', 'Plugins', 'More', 'Pinned' (with 'Random Stuff', 'Angie Project', and 'introSe'), and 'Projects' (with 'Hang Nguyen', 'Khanh Linh', 'introSe', and 'Phân tích KNN và tham số'). The main chat area, titled 'Khanh Linh', shows a conversation. The user's message is 'Và phần submit từng câu trước đó:'. The AI's response is a TypeScript code block for a `handleSubmitAnswer` function. Below this, the user asks 'DTO từng câu:'. The AI responds with a TypeScript class definition for `SubmitAnswerDto`. At the bottom, there are buttons for 'Pasted code' and 'PracticeQuestions.updated'.

ChatGPT Plus

Khanh Linh

Và phần submit từng câu trước đó:

```
« TypeScript

const handleSubmitAnswer = () => {
  const answer = answers[currentQuestion.id]?.trim();

  if (!answer) {
    showNotification(
      "error",
      "Please enter or select an answer before submitting.",
    );
    return;
  }

  console.log(currentQuestion.id, answer);

  showNotification(
    "success",
    "Answer submitted successfully.",
  );
};
```

DTO từng câu:

```
« TypeScript

export class SubmitAnswerDto {
  @IsString()
  @NotEmpty()
  questionId!: string;

  @IsString()
  @NotEmpty()
  userAnswer!: string;

  @IsString()
  @IsIn(['multiple-choice', 'written'])
  questionType!: 'multiple-choice' | 'written';
}
```

Pasted code PracticeQuestions.updated

Prompt 2

Facebook

Khanh Linh - introSe

New Tab

Ask Gemini

chatgpt.com/g/g-p-6a68a9bb684881919c6c8883ffc1d78c/c/6a69b48b-21fc-83ed-b45d-ae5ef19a12bc

Pasted code

PracticeQuestions.updated

Prompt 2 — data.questions vs data.data.questions

Đây chính là đoạn tôi đã hướng dẫn để frontend chịu được cả hai dạng response:

<> TypeScript

```
const questionList =  
  data.questions ||  
  data.data?.questions ||  
  [];  
  
setQuestions(questionList);
```

Pasted code

Prompt 3

Facebook

Khanh Linh - introSe

New Tab

Ask Gemini

chatgpt.com/g/g-p-6a68a9bb684881919c6c8883ffc1d78c/c/6a69b48b-21fc-83ed-b45d-ae5ef19a12bc

ChatGPT Plus

Scheduled

Plugins

More

Pinned

Random Stuff

Angie Project

introSe

Projects

Hang Nguyen

Khanh Linh

introSe

Phân tích KNN và tham số

Thomas Le Plus

Khanh Linh

Để làm avatar bo tròn có chữ cái đầu giống góc phải màn hình, phần chính cần sửa ở **Frontend**, không phải Backend.

Backend hiện đã trả về `name` của user, nên frontend có thể lấy chữ cái đầu từ tên, ví dụ:

<> JavaScript

```
const initial = user?.name?.trim()?.charAt(0)?.toUpperCase() || '?';
```

Những file cần xem để sửa chính xác

Bạn cần gửi các file frontend sau:

- File trang profile đang hiển thị khối lớn trong ảnh, thường có tên như:

Frontend/src/features/profile/ProfilePage.jsx

hoặc:

Frontend/src/pages/ProfilePage.jsx

- File CSS tương ứng:

Ask ChatGPT

Medium

ChatGPT Plus

Khanh Linh

ProfilePage.css

hoặc file dùng Tailwind/CSS module.

3. File component avatar ở góc phải màn hình, vì có thể tái sử dụng đúng thiết kế đó:

```
Header.jsx
Navbar.jsx
Topbar.jsx
Layout.jsx
```

4. File lấy dữ liệu user, ví dụ:

```
userService.js
authService.js
api.js
AuthContext.jsx
```

Hiện tại mình chưa có các file frontend này, nên chưa thể chỉ đúng dòng nào cần sửa.

Cách hiển thị avatar chữ cái

Trong trang profile, phần đang là vòng tròn xanh trống nên đổi thành:

+ Ask ChatGPT Medium

ChatGPT Plus

Khanh Linh

Cách hiển thị avatar chữ cái

Trong trang profile, phần đang là vòng tròn xanh trống nên đổi thành:

```
<div className="profile-avatar">
  {user?.name?.trim()?.charAt(0)?.toUpperCase() || '?'}
</div>
```

CSS:

```
.profile-avatar {
  width: 240px;
  height: 240px;
  border-radius: 50%;

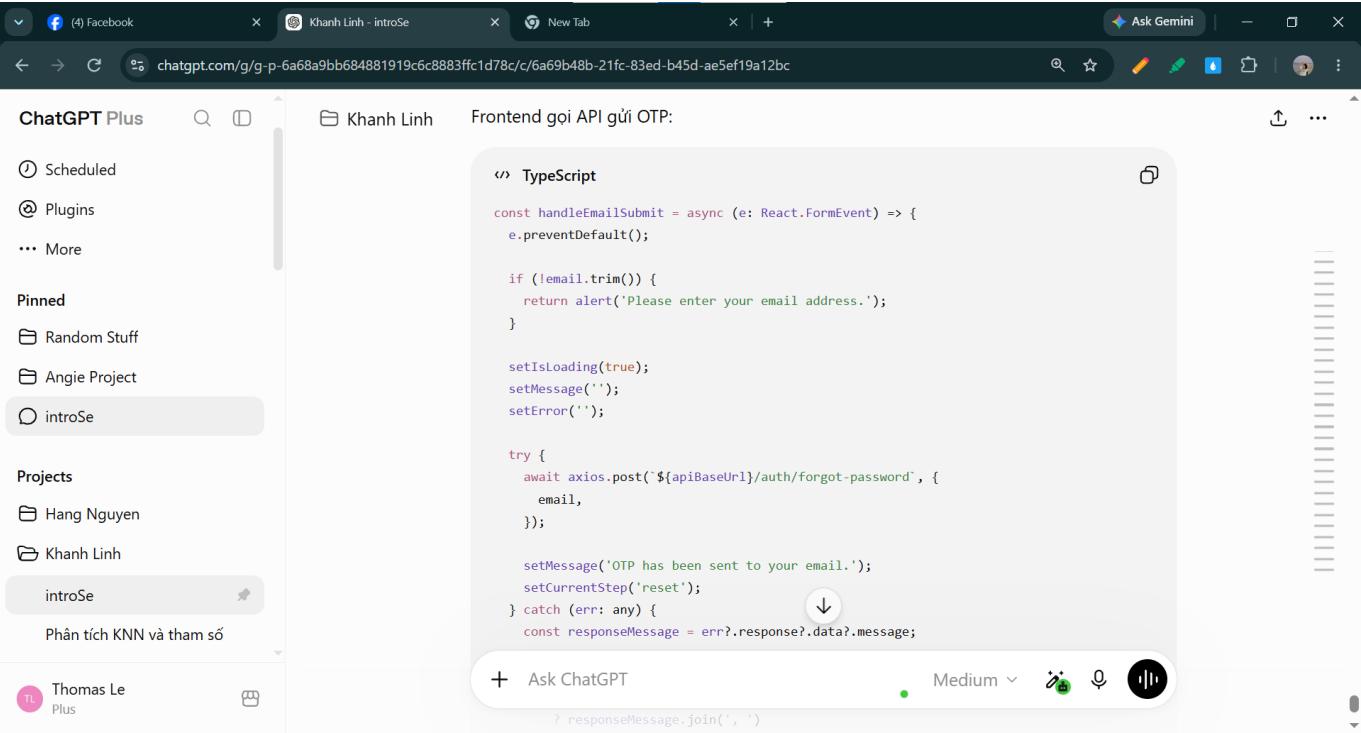
  display: flex;
  align-items: center;
  justify-content: center;

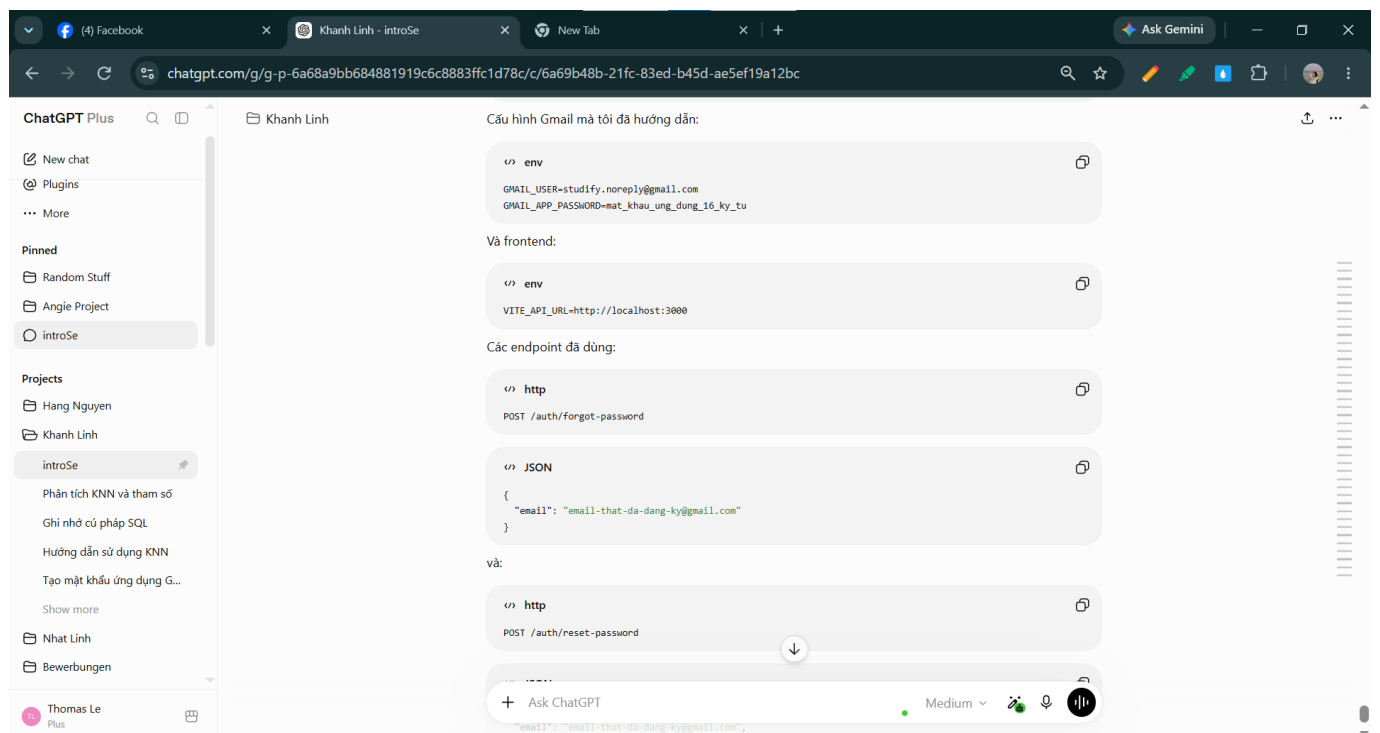
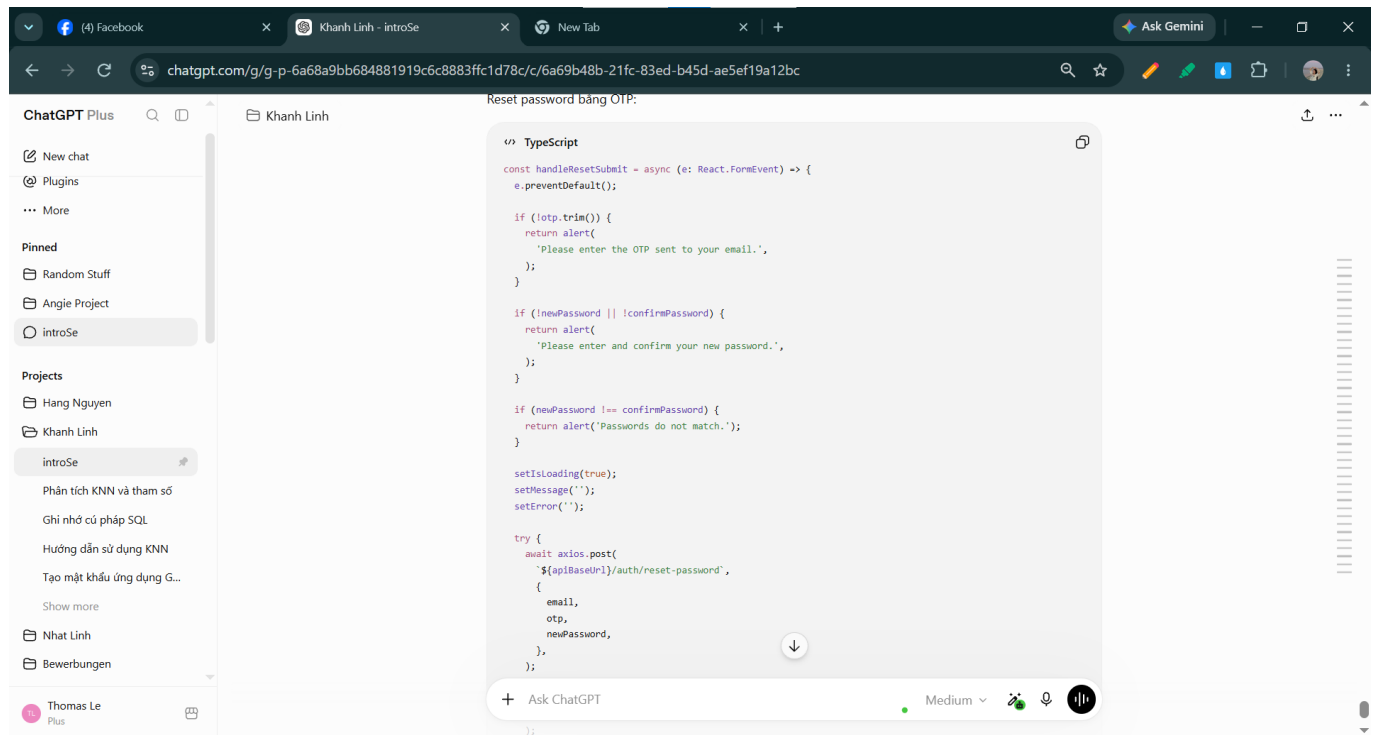
  background-color: #2563eb;

  font-size: 80px;
}
```

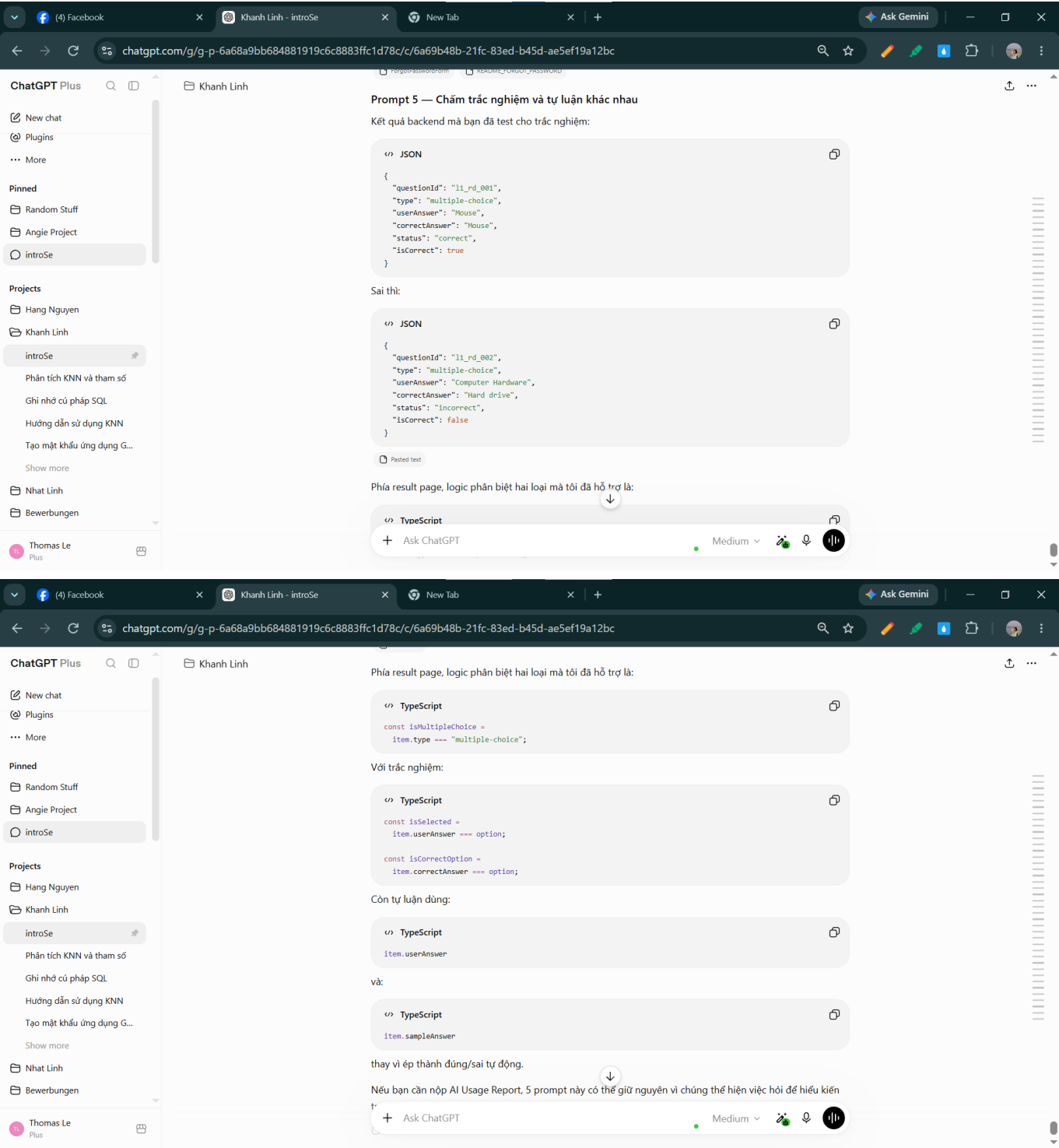
+ Ask ChatGPT Medium

Prompt 4





Prompt 5



Gia Phúc

Prompt 1

List all the prominent UI errors that can currently be seen

10:18 PM

Prominent UI issues currently visible

- **Roadmap page**
 - The roadmap panel is overlapping the fixed side menu while scrolling.
 - Scroll behavior/layout spacing is wrong, so roadmap cards get clipped under the sidebar instead of staying in view.
- **Lessons page**
 - Lesson item icons are not aligned consistently, making the list look uneven.
 - The theory card panel is missing from the lesson detail view, so the lesson overview feels incomplete.
- **Test result page**
 - The result percentage bar is not displaying correctly.
 - The progress indicator is either not updating or not rendering the score as intended.
- **Lesson card styling**
 - Font sizes and text colors are inconsistent with the design spec.
 - Some lesson card text appears too large/small or uses the wrong color contrast, reducing visual hierarchy.

Prompt 2

Phân hệ 3: Không gian Học nhóm (Virtual Study Room) giới hạn từ 2 - 5 người

F3.1: Quản lý Phòng học/Nhóm học (Group Management)
Tạo nhóm học tập mới (Hệ thống tự động cấp Mã Code nhóm).

Tham gia nhóm học tập thông qua Mã Code (Join via Code).

F3.2: Phân quyền Thành viên (Role Authorization)

Quyền Leader (Trưởng nhóm): Quản lý nhóm, giao việc, đăng tải tài liệu.

Quyền Member (Thành viên): Xem thông tin, nhận nhiệm vụ, tải tài liệu.

F3.3: Quản lý & Giao Nhiệm vụ (Task Assignment)

Đối với Leader: Tạo lịch học, tạo nhiệm vụ, đặt hạn chót (Deadline) cho nhóm.

Đối với Member: Nhận và xem danh sách nhiệm vụ cần làm thông qua một Widget thông báo hiển thị trực tiếp tại Dashboard cá nhân khi đăng nhập.

F3.4: Kho lưu trữ tài liệu chung (Shared File Repository)

Đăng tải tài liệu học tập bổ sung (Hỗ trợ định dạng PDF, hình ảnh).

Xem và tải về (Download) tài liệu do các thành viên khác chia sẻ.

F3.5: Real-time chat

Các thành viên có thể thảo luận với nhau trong một khung chat chung.

Explain the flow of going to the group's page and accessing the file repository and download/upload files



Here is the step-by-step user flow for accessing the group page and interacting with the Shared File Repository (uploading and downloading files).

Here is the step-by-step user flow for accessing the group page and interacting with the Shared File Repository (uploading and downloading files).

① **Navigate to the Group Page**

Prerequisite: User is logged in

1. From the **Personal Dashboard**, navigate to the **Virtual Study Room** section.
2. Choose how to enter:
 - **Existing Group:** Select the target study group from your joined list (2–5 members limit).
 - **New Group:** Click "Join via Code" and enter the 6–8 character Group Code provided by the Leader (F3.1).
3. You will land on the main **Group Dashboard** showing active members, real-time chat, and task widgets.

② **Access the Shared File Repository**

1. On the group navigation bar, click the "**Shared Files**" or "**Repository**" tab (F3.4).
2. The system fetches and displays the file list, including:
 - **File Name & Extension**
 - **Uploader** (Leader or Member name)
 - **Upload Timestamp & File Size**

③ Upload a File (Leader or Member)

Supported Formats: PDF, PNG, JPG, JPEG

1. Click the **"Upload File"** button at the top of the repository section.
2. Select a file from your device via the OS file picker.
3. The system checks file constraints:
 - Validates file extension (PDF or Image formats).
 - Checks storage size limits.
4. Click **"Confirm Upload"**.
5. Once processing is complete, the file appears in the group repository list, and an automatic notification is sent to the group's **Real-time Chat (F3.5)**.

④ View & Download Files

1. Locate the desired file in the **Shared File Repository** list.
2. Choose an action:
 - **Preview:** Click the file row/thumbnail to view PDFs or images directly in an in-browser modal without downloading.
 - **Download:** Click the **"Download"** button/icon next to the file to save it directly to your local storage.

④ View & Download Files

1. Locate the desired file in the **Shared File Repository** list.
2. Choose an action:
 - **Preview:** Click the file row/thumbnail to view PDFs or images directly in an in-browser modal without downloading.
 - **Download:** Click the "Download" button/icon next to the file to save it directly to your local storage.

Access & Permission Summary

Role	Access Level	Actions Allowed
Leader	Full Access	View, Upload, Preview, Download, Delete any file
Member	Standard Access	View, Upload, Preview, Download own/shared files
...		

Would you like me to outline the database schema or API endpoints for this file management module?

Prompt 3

hợp hơn
lạm dụng quan hệ <-
chưa hợp lý như "Việ
quan hệ <<extend>> ,
ing ko phù hợp có ác
le; đánh ID sai cũng

can you check if my UC mermaid model fit my professor's feedback? anything that need changing?
flowchart LR
%% Actors

No, your current Mermaid model **does not fit** your professor's feedback and still has major UML design flaws.

What's Wrong with Your Current Model

1. **Abusing <<include>> as Functional Decomposition (Sequence Workflow)**
 - Your model shows UC5.2 and UC5.3 directly including UC5.1 .
 - Having multiple sub-use cases include another sub-use case to mimic "Step 1: Speech Recognition → Step 2: Reply / Evaluate" treats Use Case diagrams like a flowchart or sequence diagram. This is the exact reason your professor criticized "*lạm dụng quan hệ <-> thành workflow*" (abusing include as a workflow).
2. **Missing a Top-Level Parent Use Case (UC5)**
 - The Learner should interact with a primary goal-oriented use case (e.g., UC5: Practice Speaking with AI).
 - Without UC5 , your sub-use cases exist in isolation without a clear umbrella interaction that triggers them during a practice turn.

2. Missing a Top-Level Parent Use Case (UC5)

- The Learner should interact with a primary goal-oriented use case (e.g., UC5: Practice Speaking with AI).
- Without UC5 , your sub-use cases exist in isolation without a clear umbrella interaction that triggers them during a practice turn.

3. Direct Actor Connections to Internal Sub-Cases

- The Learner is directly connected to UC5.1 and UC5.2 , while AI Engine is connected to every single sub-use case (UC5.1 , UC5.2 , UC5.3 , UC5.4).
- Primary actors should connect to the main use case (UC5), while internal automated sub-case interactions should not clutter actor connections.

4. Direction of <<extend>>

- Although UC54 -. "<<extend>>" .-> UC53 points in the correct UML direction (Extension → Base), UC5.4 should extend the main workflow or evaluation context where the weakness is identified, rather than dangling off a low-level sub-case without the overarching UC5 context.

How to Fix It

- **Add** UC5: Practice Speaking with AI inside the system boundary as the main Use Case.
- **Connect** Learner and AI Engine to UC5 as the primary/secondary actors.
- **Make** UC5 include UC5.1 , UC5.2 , and UC5.3 so the top-level use case encapsulates the included steps.
- **Make** UC5.4 extend UC5.3 (or UC5), triggered conditionally when performance feedback/guidance is needed.
- **Remove direct lines from actors to** UC5.1 , UC5.2 , UC5.3 , and UC5.4 to keep the diagram clean and UML-compliant.

Thiên Phước

Prompt 1

Reviewing C4 Component Diagrams

All diagrams must be drawn using Mermaid format. You may use standard Mermaid flowchart/graph syntax with C4-style labeling (Mermaid's experimental C4 syntax is not required).

For each container, provide a written description of:

- Its responsibility and the services it provides.
- The technology/framework used.
- How it communicates with other containers (e.g., HTTP/HTTPS, WebSocket, database connection).

For each component in the Level 3 diagram, describe its responsibility and its relationships with other components.

Warning: The architecture diagrams must accurately reflect your actual implementation at the time of submission. Any inconsistency between the documented architecture and the source code will result in a grade penalty.

PA4-2026 2

As you continue implementing features, you must keep the architecture document up to date.

Sau khi đọc kỹ yêu cầu và đối chiếu với file `C4 Model - Level 3 (Component Diagram).md` (cùng với ngữ cảnh từ file Level 2), mình nhận thấy file Level 3 của nhóm đang có **2 vấn đề/thiếu sót lớn** liên quan trực tiếp đến yêu cầu chấm điểm, và đặc biệt là **chưa chọn đúng chức năng quan trọng nhất** để thể hiện kiến trúc.

Dưới đây là nhận xét chi tiết để nhóm chỉnh sửa:

1. Thiếu sót về việc chọn "Chức năng quan trọng" (Feature Selection)

- Vấn đề:** Nhóm đang chọn tính năng **Authentication (Đăng nhập/Đăng ký)** để vẽ Level 3. Dù Authentication quan trọng, nhưng nó mang tính chất khuôn mẫu (boilerplate), quá cơ bản và gần như app nào cũng giống nhau. Nó **không đáp ứng tốt** tiêu chí: *"focus on the most important features that best represent the internal structure"* của hệ thống **STUDIFY**.
- Vì sao?** Nhìn vào Level 2, kiến trúc của Studify có những điểm đặc thù rất hay và phức tạp như giao tiếp **WebSocket (WSS)** và gọi **AI API (STT & LLM)**, hoặc **AWS S3**. Authentication không hề lột tả được các luồng kiến trúc này.
- Khuyến nghị (Thiếu diagram cho chức năng cốt lõi):** Nhóm **NÊN bổ sung hoặc thay thế** bằng Component Diagram cho 1 trong 2 chức năng sau:

Reviewing C4 Component Diagrams

All diagrams must be drawn using Mermaid format. You may use standard Mermaid flowchart/graph syntax with C4-style labeling (Mermaid's experimental C4 syntax is not required).

For each container, provide a written description of:

luồng kiến trúc này.

- Khuyến nghị (Thiếu diagram cho chức năng cốt lõi):** Nhóm **NÊN bổ sung hoặc thay thế** bằng Component Diagram cho 1 trong 2 chức năng sau:
 - AI Speaking Assistant (F5):** Đây là Core Feature. Biểu đồ sẽ show được cách Frontend UI tương tác với Backend, và Backend components (như `AIService`, `AudioProcessorController`) tương tác với **External AI API** (gọi STT/LLM thế nào).
 - Virtual Study Room (F3):** Chức năng này cực kỳ quan trọng để show được cách tổ chức các components xử lý **WebSocket/WSS** real-time (như `ChatGateway`, `RoomService` ở Backend và `WebSocketClient`, `RoomStore` ở Frontend).

2. Thiếu sót về Format theo Requirement (Bị thiếu Container Description)

- Yêu cầu ghi rõ:**

"For each container, provide a written description of: Its responsibility and the services it provides. The technology/framework used. How it communicates with other containers."
- Vấn đề trong file:** Hiện tại file của nhóm đi thẳng vào phần `### Component Descriptions:` (mô tả các thành phần con bên trong như `Auth UI`, `AuthStore`). Nhóm **đã quên viết phần mô tả tổng quát cho bản thân cái Container đó** trước khi đi vào Component.
- Cách khắc phục:** Dưới mỗi heading `## 1. Web Application Container...` và `## 2. Backend API Container...`, nhóm cần bổ sung một đoạn ngắn mô tả lại Container đó theo đúng 3 ý yêu cầu (Responsibility, Technology, Communication). Dù thông tin này có thể đã có ở file Level 2, nhưng đề bài yêu cầu trình bày lại ở Context của tài liệu này để chấm điểm.

3. Góp ý thêm để Level 3 Diagram chi tiết (Sâu) hơn

Vì Level 3 là mức Component, cần mô tả chi tiết "bên trong" container, các diagram hiện tại của nhóm hơi ít component (chỉ có 2-3 hộp). Để đạt điểm tối đa, nếu vẽ cho Frontend hay Backend, bạn nên chèn nhỏ thêm một chút cho đúng thực tế triển khai:

- Frontend:** Thay vì chỉ có `UI Components` và `AuthStore`, bạn có thể thêm các component như `API Client / Axios Interceptor` (chuyên xử lý attach token vào header), hoặc `React Router / AuthRouter` (chuyên hân về các private routes)

Prompt 2

trường nhóm giao là mỗi thành viên tự sửa lại phần của mình đã làm trong PA3, trong đó thì mình đã làm '' module 4 ở phần D. Bạn phân tích kỹ xem mình có cần sửa gì dựa vào feedback của thầy không?

10:40 AM

Worked for 1m

Chào bạn! Do server vừa khởi động lại nên code repo của nhóm dường như đã được rollback về trạng thái ban đầu (các module vẫn bị xé lẻ thành hàng chục file `UC_spec.md` và nằm rải rác ở các thư mục `Module_X` thay vì đã được gộp lại). Việc trường nhóm yêu cầu "ai làm phần nấy tự sửa" là một bước đi hợp lý lúc này để tránh conflict. Mình đã phân tích rất kỹ tình trạng các file thuộc **Module 4 (Study Utilities)** của bạn và đối chiếu trực tiếp với các lỗi mà giảng viên (TA) đã chỉ ra.

Sự thật là phần Module 4 của bạn đang vướng phải **2 lỗi nặng nhất** mà giảng viên đã nhắc nhở:

- Lỗi phân rã chức năng (Functional Decomposition):** Bạn đã biến các thao tác nhỏ (như "Viết giải thích", "Lật thẻ", "Thêm Tag") thành các Use Case riêng biệt và nối chúng bằng `include`. Đây không phải là Use Case chuẩn.
- Lỗi trùng lặp ID:** Bạn đang đặt tên là `UC1`, `UC2`, ... Việc này sẽ bị trùng lặp với `UC1` của các bạn khác làm Module 1, 2, 3.

Mình đã soạn sẵn một bản **Kế hoạch Sửa lỗi (Action Plan)** rất chi tiết ở màn hình bên cạnh (file `implementation_plan.md`). Trong đó mình giải thích rõ vấn đề và đề xuất gộp 10 Use Case lỗi của bạn thành **3 Use Case chuẩn mực**.

Bạn vui lòng đọc kỹ bản kế hoạch bên cạnh nhé. Nếu bạn đồng ý với hướng giải quyết này, chỉ cần nhấn **Proceed** (hoặc trả lời đồng ý) để mình bắt tay vào việc gộp file, viết lại nội dung các flow cho chuẩn và cập nhật lại luôn sơ đồ Mermaid cho bạn!

✔ Auto-proceeded with

Implementation Plan

Worked for 2m

← 0 Files With Changes

Review Changes

Ask anything, @ to mention, / for actions

+

Gemini 3.1 Pro (High)

^

35 / 39

trường nhóm giao là mỗi thành viên tự sửa lại phần của mình đã làm trong PA3, trong đó thì mình đã làm Usecase cho module 4 ở phần D. Bạn phân tích kỹ xem mình có cần sửa gì dựa vào feedback của thầy không?

Worked for 2m >

Mình đã thực hiện xong toàn bộ các bước khắc phục lỗi từ TA cho phần Module 4 của bạn. Cụ thể:

- Sửa sơ đồ Mermaid trong `report.md`:** Mình đã xóa bỏ các mũi tên `include` lạm dụng (sai chuẩn UML) và rút gọn lại thành đúng 3 Use Case độc lập.
- Gộp file `Use Case Specifications`:** Mình đã xóa bỏ 10 file nhỏ lẻ bị lỗi "Functional Decomposition" (chia nhỏ tính năng thành các thao tác lật vật) và gộp nội dung của chúng thành 3 Use Case tiêu chuẩn, mang ý nghĩa mục tiêu rõ ràng của người dùng:
 - `M4-UC1` : Create Flashcard
 - `M4-UC2` : Study Flashcards
 - `M4-UC3` : Use Pomodoro Timer
- Cập nhật Naming Convention (ID):** Các UC đều được gắn tiền tố `M4-` để không bị trùng lặp ID với các thành viên khác trong nhóm.
- Nối Prototypes:** Đã chèn đầy đủ 17 ảnh UI Prototypes (cho cả luồng cơ bản và thay thế) vào 3 tài liệu mới này.

Bạn có thể mở artifact `walkthrough.md` bên cạnh để xem chi tiết, hoặc trực tiếp vào folder `Module_4/` và file `report.md` để kiểm tra. Phần của bạn bây giờ đã hoàn toàn đạt chuẩn tuyệt đối và sẵn sàng để ghép với các bạn khác trong nhóm! Bạn có muốn kiểm tra thêm nội dung nào nữa không?

Implementation Plan

Analysis of TA feedback for Module 4 Use Cases and proposed restructuring plan to merge fragmented use cases.

Task

Task list for executing Module 4 TA feedback fixes

< 0 Files With Changes

Review Changes

Ask anything, @ to mention, / for actions

+ Gemini 3.1 Pro (High)

Minh Thư

Prompt 1

kiểm tra xem vì sao khi bắt đầu, nó bị trừ 2s? tức là từ 1 phút, nó không xuống 59s mà xuống thẳng 58s. và t muốn chỉnh sửa thêm: khi ấn refresh timer hay reload gì đấy, nó sẽ tự động quay về mode focus chứ không phải relax

Nguyên nhân bị trừ 2s khi bắt đầu

Lỗi nhảy từ 60s xuống 58s xuất phát từ 2 nguyên nhân kết hợp:

- 1. Dùng `Math.floor` thay vì `Math.ceil` : Khi vừa đếm ngược được 1.001 giây, thời gian còn lại là `58.999` giây. Công thức `Math.floor(58.999)` sẽ làm tròn xuống thẳng **58**, bỏ qua giây **59**.
- 2. Độ trễ của `setInterval(1000)` : Hàm `setInterval` chạy sau mỗi 1000ms. Do vòng lặp sự kiện (event loop) của JavaScript, lần chạy đầu tiên thường mất khoảng `1005ms` — `1010ms`, khiến thời gian còn lại luôn `< 59` giây ngay ở lần đếm đầu tiên.

Các chỉnh sửa đã áp dụng

- 1. Sửa lỗi nhảy 2s:
 - o Thay `Math.floor` bằng `Math.ceil` để `58.999s` làm tròn đúng thành `59s`.
 - o Giảm chu kỳ `setInterval` từ `1000ms` xuống `200ms` giúp đồng hồ cập nhật chính xác từng miligiây, không bị trễ nhịp.
- 2. Chuyển về mode `focus` khi Reset / Reload:

Prompt 2

```
}, [timeLeft, isRunning, mode]); cái này, t muốn khi người  
dùng di chuyển sang tab khác trong studify (vdu: đường dẫn  
hiện tại là /dashboard/pomodoro, khi user di chuyển qua  
đường dẫn /dashboard thì vẫn sẽ hiện giống vậy)
```

Vấn đề này xảy ra do **Vòng đời Component (Lifecycle)** trong React:

Khi bạn chuyển route từ `/dashboard/pomodoro` sang `/dashboard`, component UI chứa hook `usePomodoroTimer` bị **unmount** (tháo khỏi DOM). Lúc này:

1. Hàm cleanup `return () => { document.title = "Studify"; }` sẽ tự động chạy và khôi phục tiêu đề về mặc định.
2. `setInterval` đếm ngược cũng bị ngắt vì component không còn tồn tại trên giao diện.

Giải pháp: Chuyển Hook thành React Context (Global State)

Để Timer vẫn đếm ngược ngầm và giữ tiêu đề Tab khi chuyển giữa các trang trong `/dashboard`, bạn chỉ cần bọc `usePomodoroTimer` vào một **React Context** ở cấp Layout chung (`DashboardLayout` hoặc `App.tsx`).

Prompt 3

Uncaught SyntaxError: The requested module
'/src/features/pomodoro/PomodoroTimer.ts' does not provide
an export named 'default' (at App.tsx:26:8)

Lỗi này xảy ra do trong `App.tsx` bạn đang import theo dạng **Default Import** (`import Pomodoro from ...`), nhưng trong file `PomodoroTimer.tsx` (hoặc `.ts`) bạn lại xuất ra bằng **Named Export** (`export const ...`) mà không có `export default` .

Bạn có thể khắc phục bằng 1 trong 2 cách sau:

Cách 1: Thêm `export default` vào file `PomodoroTimer` (Khuyến dùng)

Mở file `src/features/pomodoro/PomodoroTimer.tsx` (hoặc `.ts`) và thêm dòng `export default` ở cuối file:

TypeScript



```
// Cuối file PomodoroTimer.tsx
export default PomodoroTimerCoreSection;
// (Hoặc tên component chính của bạn trong file đó, ví dụ: export default PomodoroTi
```